

МФТИ
Алгоритмы и структуры данных I, осень 2025
Программа экзамена

Всюду, где уместно и не сказано иное, пункт программы подразумевает формулировку решаемой задачи, описание алгоритма, доказательство его корректности и анализ асимптотики. Факты, не требующие доказательств на экзамене, помечены «б/д».

1. RAM-модель вычислений: допустимые операции. Измерение времени работы алгоритма в терминах асимптотики: пример.
2. Асимптотические обозначения: O , Ω , Θ . Независимость от стартового индекса. Примеры.
3. Сумма на отрезке в статическом массиве: префиксные суммы.
4. Проверка вхождения числа в отсортированный массив: бинарный поиск.
5. Задача сортировки. Две постановки, их эквивалентность.
6. Доказательство формулы: $\log(n!) = \Theta(n \log n)$.
7. Нижняя оценка на число сравнений в сортировке сравнениями.
8. Сортировка слиянием (Merge Sort).
9. Решение рекуррентности $T(n) = 2T(n/2) + O(n)$.
10. Подсчёт числа инверсий в массиве.
11. Быстрая сортировка (Quick Sort). Асимптотика — б/д.
12. Поиск k -й порядковой статистики с выбором случайного пивота (Quick Select). Асимптотика — б/д.
13. Дерандомизированный алгоритм поиска k -й порядковой статистики за $O(n)$, где n — длина массива.
14. Дерандомизированный алгоритм быстрой сортировки за $O(n \log n)$, где n — длина массива.
15. Стабильная сортировка подсчётом. Сортировка пар чисел.
16. Цифровая сортировка (LSD).
17. Односвязный, двусвязный списки. Добавление, удаление элементов.
18. Структура данных «стек»: реализация списком.
19. Проверка правильности скобочной последовательности с несколькими типами скобок.
20. Поиск ближайшего меньшего/большого слева/справа в статическом массиве.
21. Вычисление значения выражения, заданного в обратной польской записи.
22. Поддержка минимума в стеке.
23. Структура данных «очередь»: реализация списком.
24. Реализация очереди на двух стеках. Асимптотика операций. Поддержка минимума в очереди.
25. Бинарная куча: определение и представление в массиве. Требование кучи.
26. Операции `siftUp` и `siftDown` в бинарной куче с доказательством корректности.
27. Выражение `insert`, `getMin`, `extractMin` и `decreaseKey` через `siftUp` и `siftDown` в бинарной куче.
28. Построение кучи (`heapify`) за линейное время (сходимостью ряда можно пользоваться б/д).
29. Сортировка кучей с привлечением $O(1)$ дополнительной памяти (Heap Sort). Несуществование кучи (основанной на сравнениях), обрабатывающей `insert` и `extractMin` за $O(1)$.
30. Удаление из кучи по указателю.
31. Удаление из кучи по значению.
32. Биномиальное дерево, биномиальная куча: определение. Преимущество перед двоичной кучей.
33. Операции `merge`, `insert`, `getMin`, `extractMin` и `decreaseKey` в биномиальной куче.
34. Амортизационный анализ, учётное время работы: определение.
35. Метод монеток (бухгалтерский учёт). Пример использования: очередь на двух стеках.
36. Структура данных «вектор» (динамический массив), реализация на массиве и оценка асимптотики.
37. Sparse Table: модельная задача, построение за $O(n \log n)$, ответ на запрос за $O(1)$.

38. Дерево отрезков: модельная задача. Обработка запросов с доказательством времени работы.
39. Дерево отрезков: двоичный спуск, поиск подотрезка с суммой $\geq x$.
40. Дерево отрезков, отложенные операции: присваивание на отрезке, операция `push`.
41. Количество чисел на отрезке, значения которых не превосходят x : Fractional Cascading.
42. Динамическое дерево отрезков.
43. Сжатие координат для дерева отрезков.
44. Персистентное дерево отрезков.
45. Количество чисел на отрезке, значения которых не превосходят x : решение с персистентным деревом отрезков.
46. Поиск k -й порядковой статистики на отрезке: решение с персистентным деревом отрезков и параллельным спуском по двум его версиям.
47. Дерево Фенвика: классическая задача, операции `update` и `getSum`.
48. Обобщение дерева Фенвика на большие размерности. Изменение асимптотики.
49. Обратное дерево Фенвика: максимум на отрезке и изменение (увеличение) в точке (`update` — без реализации).
50. Дерево Фенвика деревьев Фенвика.
51. Дерево поиска: определение и операции (без реализации) `find`, `insert`, `erase`, а также опциональные `merge` и `split`. Запросы, использующие порядок на элементах.
52. AVL-дерево: определение. Оценка глубины AVL-дерева на n вершинах.
53. Устранение дисбаланса в AVL-дереве.
54. AVL-дерево: реализация операций `find`, `insert` и `erase`.
55. Splay-дерево: определение и практическая значимость.
56. Splay-дерево: операции `zig`, `zig-zig` и `zig-zag`, операция `splay`.
57. Метод потенциалов.
58. Амортизированное время работы операции `splay` с помощью метода потенциалов.
59. Splay-дерево: реализация `insert`, `erase` и `find`, связь с операцией `splay`, оценка времени работы (б/д).
60. Splay-дерево: реализация `merge` и `split`, связь с операцией `splay`, оценка времени работы (б/д).
61. Дерево поиска по неявному ключу: в массиве вставить, удалить, изменить элемент, нахождение суммы на отрезке.
62. Хеш-функции. Коллизии. Универсальное и k -независимое семейства хеш-функций. Примеры таких семейств.
63. Хеш-таблицы с цепочками. Реализация операций `find`, `insert`, `erase`. Процедура `rehash`.
64. Совершенное хеширование.
65. Хеш-таблицы с открытой адресацией. Реализация операций `find`, `insert`, `erase`. Процедура `rehash`. Теорема о времени работы для линейного пробирования, двойного хеширования.
66. Фильтр Блума. Алгоритм и оптимальные значения параметров (б/д).
67. Фильтр кукушки (б/д).
68. Skip List. Операции `find`, `insert`, `erase`. Асимптотика — б/д.
69. Декартово дерево: определение и теорема о глубине (б/д). Построение за линейное время (при условии, что ключи отсортированы).
70. Реализация операций `merge` и `split` в декартовом дереве.
71. Выражение `insert` и `erase` в декартовом дереве через `merge` и `split`.
72. В-дерево: определение и практическая значимость. Реализация операции `find`.
73. Оценка глубины В-дерева на n ключах при фиксированном параметре t .
74. Реализация операции `insert` в В-дереве.
75. Реализация операции `erase` в В-дереве.
76. Красно-чёрное дерево: определение.
77. Оценка глубины красно-чёрного дерева на n ключах.
78. [Можно пользоваться официальной шпаргалкой с разбором случаев] Реализация операции `insert`

в красно-чёрном дереве.

79. [Можно пользоваться официальной шпаргалкой с разбором случаев] Реализация операции **erase** в красно-чёрном дереве.
80. Сравнительный анализ различных реализаций дерева поиска: наивное, AVL-, splay-, B-, декартово и красно-чёрное дерево.